

Lecture 04 — Functions — a named promise

Algorithmic Thinking · Week 2, Session 4 — Friday 2 October 2026, 17:35–18:55
(lab 16:30–17:30 immediately before)

Leopoldo Rodríguez Salazar · Tecnológico de Monterrey

Contents

Overview	1
Opener — Sorting, and the function you never wrote (10 min)	1
Notes	2
Stretch — if the session was easy	5
After the session	5

Objective. Write a function with parameters and a return value, call it from a main program, and explain the difference between returning a value and printing one.

Overview

You have called functions since your second program: `float()`, `print()`, `round()`, `math.sqrt()`. Someone wrote each of them once, gave it a name, and you have been using the name ever since without reading a line of what is inside. Today you write your own.

One idea: **a function is a named promise — inputs in, value out.** `def` makes the promise, `return` keeps it, and calling the function by name is how the rest of the program collects. The hard part is not the syntax. It is the difference between **returning** a value and **printing** one — the single most confusing idea of the first month, and the one on which every program from week 3 onwards depends.

Opener — Sorting, and the function you never wrote (10 min)

Algorithms to Live By, chapter 3. Hand out five playing cards, face down, to a pair in the front row. “Sort them, smallest to largest — and tell me *exactly* what you are doing, step by step, so that the person next to you could do it with their eyes closed.”

What they describe is almost always **insertion sort**: take the next card, slide it left until it is in place. It is a fine algorithm for five cards. For a million it is a disaster: the work grows with the *square* of the number of cards. **Merge sort** — split, sort the halves, merge — grows with $n \log n$, and that difference is why your phone can sort a million contacts before you lift your finger. It is also why sports tournaments are brackets, why a library is shelved alphabetically only where people will search, and why the book’s advice

is: *sorting something you will never search is a waste; searching something you never sorted is merely inefficient.*

Now the point for today. In Python:

```
>>> sorted([9, 4, 1, 7, 2])
[1, 2, 4, 7, 9]
```

Somebody wrote that — Tim Peters, 2002, an algorithm called Timsort — **once**. Gave it a name. Since then every Python programmer on earth has *called* it, by name, with an input, and received an output, without knowing or caring what is inside. **That is a function**. Today you make your own: something you would otherwise write three times.

Your turn, 3 minutes, on paper: write your card-sorting procedure as an algorithm. INPUT: five cards. OUTPUT: the same cards, in order. Numbered steps. Then draw a box around the whole thing and give it a **name**. You have just written `sort_cards`.

Notes

The smell: the same lines three times

Lab 1's triangle program, then Homework 1, then today's lab: every one of them read a number, converted it, checked it, computed something. Look at `01_repeated_code.py`: three areas of three triangles, and the formula typed three times. Change the formula — say the base is now in centimetres and you want metres — and you change it in three places, and you will miss one. That is not a style problem. It is a *correctness* problem: **code that exists twice will one day be right in one place and wrong in the other**.

def — make the promise

```
def triangle_area(base, height):
    area = base * height / 2
    return area
```

Line by line: `def`, a **name** (same rules as variables), **parameters** in parentheses — the names this function will use for its inputs — and a colon. Then the body, indented, like an `if` block. And `return`, which hands `area` back to whoever called and ends the function.

Nothing has happened yet. Defining a function is like writing a recipe: no food is cooked until someone follows it. `02_first_function.py` — run it as written and it prints nothing.

Calling it — collect on the promise

```
a = triangle_area(10, 5)      # 25.0 goes into a
print(a)
print(triangle_area(3, 4))    # 6.0 — the call IS a value; use it anywhere a value fits
```

10 and 5 are **arguments**; they land in `base` and `height` **in order**. The call `triangle_area(10, 5)` is an expression with a value, exactly like `5 > 3` or `d / t` — you can store it, print it, add it, compare it, pass it to another function.

Gloss: *función · definir* — define · *parámetro* — parameter · *argumento* — argument · *llamar* — to call · *devolver / regresar* — return · *alcance* — scope

return versus print — the whole idea

Two functions that look the same and are not:

```
def area_returns(base, height):  
    return base * height / 2
```

```
def area_prints(base, height):  
    print(base * height / 2)
```

```
>>> x = area_returns(10, 5)
```

```
>>> x
```

```
25.0
```

```
>>> y = area_prints(10, 5)
```

```
25.0
```

```
>>> y
```

```
>>> print(y)
```

```
None
```

area_prints **showed** 25.0 to you and gave the program **nothing** — y holds None. Try area_prints(10, 5) + 1 and you get TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'. The function did its job for a human and is useless to the rest of the program. 03_return_vs_print.py.

The rule, from today until December: **functions return; main prints**. A function that computes something *returns* it. Displaying is a separate job, done in one place.

Scope — what happens in the function stays in the function

```
def triangle_area(base, height):  
    area = base * height / 2  
    return area
```

```
result = triangle_area(10, 5)
```

```
print(area)          # NameError: name 'area' is not defined
```

area was created inside the function and ceased to exist when the function returned. The only things that cross the boundary are the arguments going in and the returned value coming out. That is a feature: it means you can read triangle_area on its own and know everything about it. 04_scope.py. (Python does let a function *read* outside names; we will not use that — it is exactly the habit that makes programs impossible to test.)

main() — the shape of every program from now on

```
def triangle_area(base, height):  
    return base * height / 2
```

```
def main():  
    base = float(input("Base: "))  
    height = float(input("Height: "))  
    area = triangle_area(base, height)  
    print("Area:", area)
```

```
main()
```

Three parts: the functions (each one a promise, each one returning), a `main()` that talks to the user and calls them, and the last line that starts it. Reading, printing and `input()` live in `main`; arithmetic and decisions live in functions. `05_main.py`.

Why bother, for twelve lines? Because in week 3 `main` grows a loop, in week 7 it grows a list, in November it grows a menu — and the functions underneath do not change. The situación problema's final program is *specified* this way: `estadistica_descriptiva(...)`, `grafical(...)`, `menu()`, `main()`. You are writing its skeleton today.

Functions that decide

Yesterday's conditional, inside a function, becomes reusable:

```
def speed_label(speed):
    if speed < 40:
        return "slow"
    elif speed < 90:
        return "normal"
    elif speed <= 120:
        return "fast"
    else:
        return "too fast"
```

Four returns, one per branch — and each one ends the function, so no `elif` is ever reached after a `return`. Test it in the shell without any `input()` at all: `speed_label(39.9)`, `speed_label(40)`, `speed_label(120)`, `speed_label(120.1)`. **That is the second reason functions exist: you can test the logic without typing inputs every time.** `06_speed_label.py`.

Failure-first: the four ways this goes wrong on Tuesday's quick exam

1. **Forgetting to call.** `def` and nothing else — the program prints nothing, and the student says “it doesn't work”. It works; nobody asked it to.
2. **print where return was needed.** Then `total = f(x) + 1` crashes with `NoneType`.
3. **Using a parameter name outside the function** — `NameError`. What goes in goes in through the parentheses.
4. **input() inside the function.** It runs, and it cannot be tested or reused — every call stops to ask. Inputs belong in `main`; values belong in parameters.

Exercise 1 — Three functions, one main (individual, 15 min)

Take Lab 1's three programs and turn them into three functions in **one** file:

```
def triangle_area(base, height): ...
def dollars_to_pesos(dollars, rate): ...
def fahrenheit_to_celsius(f): ...
```

Each returns; none prints; none calls `input()`. Then a `main()` that reads the values for all three, calls them, and prints the three results. Tests are the same as Lab 1: `25.0`, `1850.0`, `100.0`. Save as `A_Matricula_f1.py`.

Before you write main, test each function **from the Thonny shell** — run the file, then type `triangle_area(3, 4)` in the shell. If it prints `6.0`, the promise holds.

Exercise 2 — A function that decides (pairs, 15 min)

Write `classify_triangle(a, b, c)` that **returns** one of "not a triangle", "equilateral", "isosceles", "scalene". Three sides make a triangle only when all are positive and each pair sums to more than the third. Algorithm on paper first — the order of the checks is the algorithm.

Tests, from the shell before any main:

call	expected
<code>classify_triangle(3, 3, 3)</code>	equilateral
<code>classify_triangle(3, 3, 5)</code>	isosceles
<code>classify_triangle(3, 4, 5)</code>	scalene
<code>classify_triangle(1, 2, 3)</code>	not a triangle
<code>classify_triangle(0, 4, 4)</code>	not a triangle

Then a `main()` that reads three sides and prints the label. Save as `A_Matricula_f2.py`.

The 1, 2, 3 row is the one that catches people: $1 + 2$ is not *greater* than 3. Strict inequality, and the algorithm has to say so.

Stretch — if the session was easy

Optional, ungraded, harder than they look.

1. **Count the comparisons.** `descending(a, b, c)` — what is the *minimum* number of comparisons that always suffices to order three numbers? Write a version that never uses more than that, and argue (on paper) why fewer is impossible.
2. **Exhaustive.** Write `check()` that calls `classify_triangle(a, b, c)` for every integer triple with sides 0-10 and counts how many land in each label. What fraction are "not a triangle"? Is there any triple where your function and a friend's disagree?
3. **A function that takes a function.** `apply_twice(f, x)` returns `f(f(x))`. Try it with `fahrenheit_to_celsius`. What does the result *mean*, and why is that a warning about what a function's parameters promise?

After the session

Homework 3 — assignments/hw-03-functions.md. Due Sunday 4 October, 23:59, on Canvas, together with Homework 2. It is short by design: rewrite your two Homework 2 programs as functions with a `main()`, and add one new function. The point is the refactor.

Also before Tuesday:

1. **Levelling module** — all ten lessons by **Thursday 8 October**; the final diagnostic is Friday 9 October in the lab (1 %).
2. **Quick exam 2 — Tuesday 6 October, first 20 minutes:** conditionals and functions. Read / predict / fix, plus one function to write. Rubric on the site.

3. **Self-study: repetition statements** (Canvas) — ten minutes, so while is not new on Tuesday.